Table 19. Reference architecture components and candidate technologies

ID	Component	Function	Candidate open technologies	Reference implementation
C1	Source connectors	Harvest public documents on a schedule, respecting site terms; record provenance	Python workers, Scrapy, cron or Airflow	Planned
C2	Ingestion and normalisation	Extract text from PDF and HTML, Bangla OCR, Unicode normalisation, chunking	Apache Tika [56], Tesseract (ben, eng) [57], custom normaliser	bapil.text, bapil.corpus
C3	Document store and catalogue	Originals, extracted text and DCAT-aligned metadata with versioning	PostgreSQL [58], MinIO (S3-compatible), CKAN [59]	JSONL corpus with checksums
C4	Search index	Lexical BM25 plus dense vectors with hybrid ranking	OpenSearch [60], or PostgreSQL with pgvector [61]	bapil.search (BM25, bilingual)
C5	Answer service	Retrieve, cite, abstain; optional LLM generation with citation verification	Open-weight LLMs served with vLLM or llama.cpp	bapil.answer (extractive, cited)
C6	Analytics engine	Budget extraction, indicators, topic models	Python, pandas, DuckDB	Planned
C7	Portal and dashboards	Bilingual, accessible user interface	Web framework; Apache Superset	Planned
C8	API gateway	Versioned REST API, quotas, rate limits	FastAPI, Kong	bapil.api (standard library)
C9	Identity and access	Single sign-on, role-based access, MFA	Keycloak	Planned
C10	Audit and observability	Tamper-evident logs, metrics, tracing	OpenTelemetry, Prometheus, Grafana	bapil.audit (hash-chained JSONL)
C11	Model registry and evaluation	Model cards, versioned evaluations, benchmarks	MLflow	bapil.evaluate
C12	Governance workbench	AI system register, risk assessments, incidents, transparency register	Lightweight web application	bapil.risk, bapil.registry

Technologies are candidates, not mandates; final choices are made by the Technical Architecture Board against requirements NFR-02, NFR-03 and NFR-06.

Source: BAPIL Technical Blueprint, Version 1.0 (2026). CC BY 4.0.